LAB 4: Introduction to the GPU

Computer Graphics 2025-2026

1 Introduction

The new important classes you should check for this lab are the following:

- **Mesh:** It's not a new class for you, but it's important that you check out the new methods: `CreateQuad` and `Render`.
- **Shader:** Class to encapsulate the default rendering pipeline; from shader creation to compilation and each of the OpenGL calls.
- **Texture:** This class represents the container for images inside the GPU. When rendering, they must be bound to the GPU.

2 Assignment

You will have to code different basic shaders to prove that you have gained the required competences to continue to the next lab, where you will implement 3D lighting shaders. Each task must be done in the same project, so the user must be able to switch between them using the input events (i.e using the keyboard or mouse), **not commenting lines of code or using the console**.

2.1 Creating a quad

To get used to coding in GLSL, your first tasks will be generating the proposed 2D images based on mathematical functions. Hence, the first step is to create a quad (using the `CreateQuad` function from Mesh) that will cover the entire screen. **You do not need to attach the mesh to an entity. Instead, store it directly in your application.**

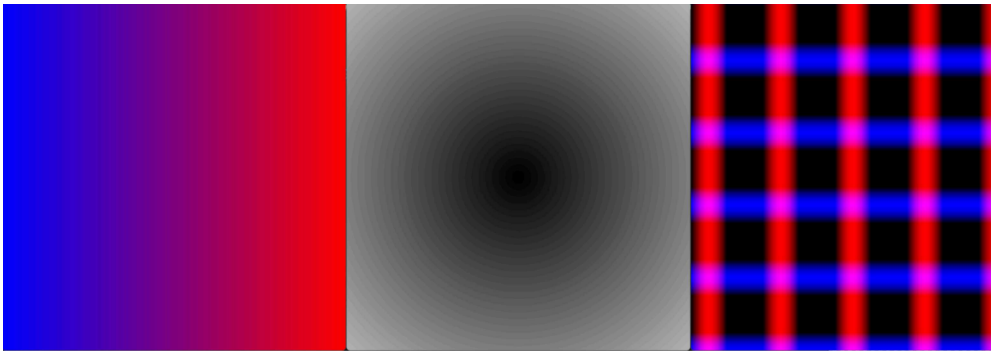
```
void Application::Render()
{
    shader->Enable();
    mesh->Render();
    shader->Disable();
}
```

2.2 Drawing formulas (1.5 points)

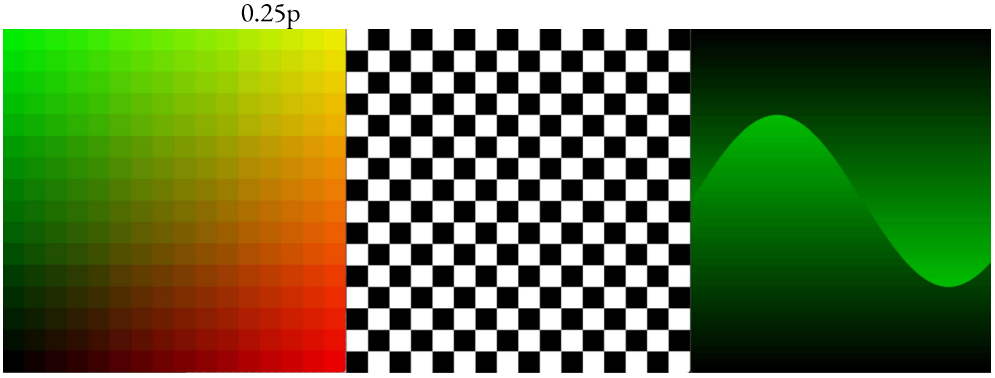
For this task, you need to recreate this set of images using mathematical formulas. The results do not need to be identical to the ones proposed (e.g. the number of blue/red columns does not matter). Consider using the aspect ratio as a parameter to avoid deformations. The following GLSL functions might be of interest:

- **mix(a, b, factor):** linear interpolation between a and b
- **distance(a, b):** distance from a to b
- **mod(a, b):** module
- **sqrt(a):** square root of **a**
- **sin(a), cos(a)**
- **floor(a):** nearest integer (less or equal to **a**)
- **step(edge, x):** 0 if x < edge, 1 otherwise

Important: You **CANNOT** use conditionals to create the formulas in this task (if clauses, switches, etc.), except for switching between tasks.



a) 0.125p b) 0.125p c) 0.25p



d) 0.25p e) 0.25p f) 0.5p

GPU). **Check the slides carefully about how to upload and sample a texture**. The following ideas may help you to achieve the task:

Important: You **CANNOT** use conditionals to create the filters in this task (if clauses, switches, etc.), except for switching between tasks.

Original image

a) 0.125p

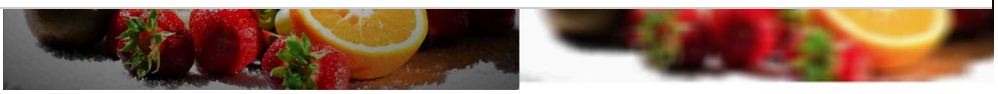
b)

0.125p

c) 0.25p

d)

0.25p



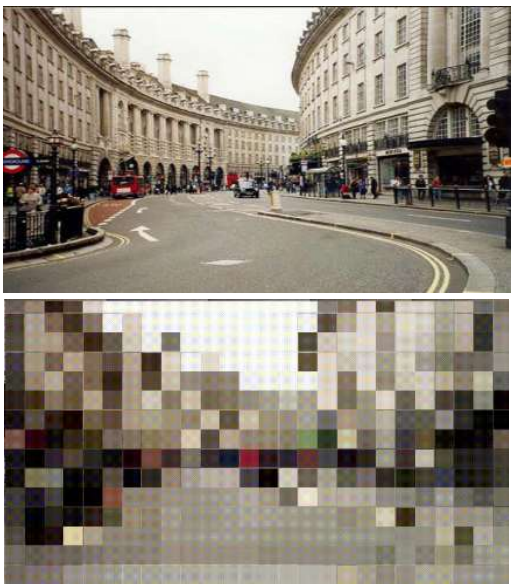
e) 0.25p

f) 0.5p

2.4 Image transformations (1 points - 0.5 each)

In this task you will have a bit of freedom. You must transform an image in **2 different ways.** From here, you can use whatever transformations you want. **All transformations must use the time variable.** E.g:

- **Rotate the image:** To rotate the image you will need to have in mind trigonometry concepts!
- **Pixelization:** The idea is to simulate a lower resolution image
- **Warping:** Any type of image distortion
- **Any other:** Be creative!



Example of pixelization

2.5 Render a 3D mesh using the GPU (1 points)

In this exercise you will **stop rendering a quad**. The objective now is rasterizing the meshes of the previous lab with their corresponding colour textures on **the GPU.**

The first step is creating the shaders that will be in charge of the raster process (i.e. “raster.vs” and “raster.fs”). You can duplicate the “simple.vs” and “simple.fs” shaders and delete the unnecessary

(texture coordinates) and the ones coming from the CPU (the texture).

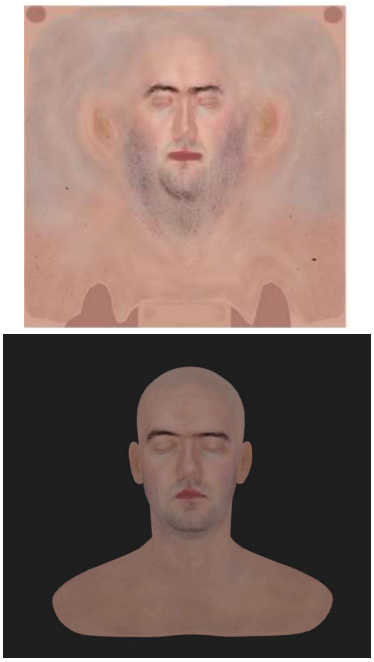
You **must** use your entity class in this task. Since your current `Entity::Render` is projecting vertices as we did in the previous lab, you have to override the method (create a new one) with the camera as the only parameter, where the only thing to do is render the entity mesh:

```
void Entity::Render(Camera* camera);
```

You will notice that you need the shader within the "Render" method of the "Entity." It's a good time to add it as an attribute to the entity, thereby simulating that this shader will be its "material."

Note: Don't forget to set the flag `GL_DEPTH_TEST` when rendering to enable occlusions.

The result of this task should be almost identical to the result obtained in the previous lab.



(a) Diffuse texture color
(b) Rendered mesh after texturing

3 Interactivity

Here are listed the KEYS with their corresponding ACTION to allow for the user to have some kind of interaction with your application. You **MUST** use these keys and no others are permitted.

KEYS	ACTION

4 Common errors

- Use while loops or std::cin that crash the application.
- Load an image/texture/shaders from disk on the `Render()` or `Update()` methods: it means you are loading it approx. 60 times per second. Loading images is super slow, you must preload them at initialization methods.
- Uniforms have different names in CPU and in the shader: When you upload data to the GPU, you must give a variable name (e.g. "u_camera_position"), that **must** be equal to the name of the variable in the shader (e.g. uniform vec3 u_camera_position)
- Confusing **texture2D** with **sampler2D**: In GLSL, a sampler2D is the variable type to store a texture with its sampling configuration. To sample it we use texture2D, a method to fetch the pixel color given a texture coordinate.
- Confusing the **clip space** with the **texture space** range: The clip space uses the range [-1, 1] while the texture space (e.g. texture coordinates) use the [0,1] range.
- Upload CPU variables as uniforms to the **vertex shader** and pass them to the **fragment shader** as varying: Uniforms can be read at any shader stage. Be sure you do not ask for extra steps for the CPU.
- Not using class operators: Remember you can simplify a lot your GLSL code by using class operators.
- It is not possible to test all tasks without changing the code (no interactivity).
- The quad is not rendered correctly: It has to be adjusted to the window dimension even after resizing.
- The custom Image framebuffer is still in use: The GPU is in charge of the rasterization, so our previous framebuffer has to be **deleted**.
- Meshes are deformed when resizing the window: The GPU resizes its framebuffer automatically, but the camera has to be updated since the aspect ratio has changed.

4 Delivery

This is the first part of Assignment 3. It will be delivered together with Lab 5.

LAB 4: Introduction to the GPU